From Coloured Points to Printable Textured Meshes

Anas Khan MBZUAI anas.khan@mbzuai.ac.ae CV802 Sub-project 2: Surface Extraction



Figure 1. Best mesh per method on VGGSfM: *textured* (our photo projection) — *shaded*. Red faces are back-facing (inconsistent orientation).

		VGGSfM (main input, 139k pts)				MVS (487k pts)		MVS-clean		
Method	Faces	CD %↓	F@1%↑	F@0.5%↑	NC↑	CD %↓	F@1%↑	CD %↓	F@1%↑	WT
Poisson (ours)	1.02M	**0.149**	**0.999**	**0.988**	0.75	0.127	0.999	0.127	0.999	
IER	17k	0.187	0.998	0.975	0.82	0.120	**1.000**	0.123	**1.000**	
Point2Mesh	10k	0.553	0.844	0.530	0.80	3.977	0.133	1.298	0.387	✓
MeshAnything V2	1.4k	0.295	0.987	0.858	0.82	0.518	0.873	0.438	0.908	
DHSP	6.0k	0.583	0.901	0.407	**0.89**	0.636	0.900	0.627	0.916	✓
NKSR*	2.09M	0.156	0.998	0.973	0.75	**0.080**	**1.000**	**0.080**	**1.000**	
SAP*	64k	0.195	0.998	0.961	0.81	0.197	0.963	0.239	0.948	✓
2DGS*	657k	0.289	0.972	0.864	0.75	0.308	0.938	0.336	0.928	

Table 1. Best run per method vs. its input cloud. CD: Chamfer-L_1, % of bbox diagonal; F@τ: F-score at τ of the diagonal; NC: normal consistency; WT: watertight (VGGSfM). * Beyond the course list. DHSP: full pipeline.

1 Task and inputs

We turn coloured points from Sub-project 1 into textured `.obj` meshes with eight methods and our own photo texturing, browsable side by side in our three.js mesh viewer (bonus; `run_viewer.sh`, CPU only). Inputs are the light-shirt capture as a VGGSfM cloud (139k points, cleaned with person masks; main input), a COLMAP MVS cloud (487k points, with normals), and the MVS cloud with 404 floating outliers removed (MVS-clean). A shared finalizer adds UV atlas, texture, metrics and renders to every mesh.

2 Method

Screened Poisson (ours, no API). Normals: PCA on 24 neighbours, oriented along a minimum spanning tree (cost $1-|\mathbf{n}_i\cdot\mathbf{n}_j|$), flipped outward per component. Inward normals are splatted into a field $\mathbf{V}$ on a 384^3 grid, and the indicator χ solves

$$\big(-\Delta+\beta S^\top W S\big)\chi = -\nabla\cdot\mathbf{V}+\beta S^\top W c, \qquad (1)$$

where S samples χ at the points and the screening term pins it to the iso-value c. We use Jacobi-preconditioned CG, coarse to fine ($96^3\to384^3$), our own marching tetrahedra (crack-free), and trim faces more than two voxels from the data: 1.0 M faces in 39 s.

Making the course methods work on a real scan. **IER**: points farthest-point sampled to its training density (12.5k).

MeshAnything V2: raw points give F@1% 0.51; our Poisson mesh fed through the authors' marching-cubes input path gives 0.987. **Point2Mesh**: the paper's `--unoriented` loss, 0.65→0.84. **DHSP**: the 3D prior is crumpled (NC 0.65) and OptCuts did not converge in 15 h. Re-meshing first (3k/6k faces) makes OptCuts converge in seconds, so the full pipeline (2D geometry prior, refinement, 2D texture prior) completes: NC 0.89, own colour.

Extra methods and texture. NKSR, Shape As Points and 2DGS (masked photos, TSDF mesh). Shape-only meshes get an xatlas atlas; each texel is rasterised to a surface point and coloured from the photos with COLMAP cameras, z-buffer visibility, person masks, weight $(\cos^4\theta\,(d/z)^2)^8$ and per-photo exposure gain (4096^2). Metrics (Tab. 1) compare each mesh with its input cloud.

3 Findings

(1) Careful classical code is competitive: our Poisson ties the best learned methods on VGGSfM; NKSR wins on dense MVS. **(2)** Point scores hide surface quality: DHSP's prior has F@1% 0.999 but is crumpled; IER's winding is inconsistent. **(3)** Input domain and outliers dominate: MeshAnything needs mesh-like input; Point2Mesh collapses on raw MVS (0.13) and recovers after floater removal (0.39). **(4)** Photo texture is far sharper than point colours. **(5)** SAP, DHSP and Point2Mesh are watertight and exported as 180 mm STL.